AI/ML Resources Hub

Overview

This page introduces common ways to run large language model (LLM) workloads on Sockeye. It focuses on practical workflows for inference, model serving, and related setup considerations, including GPU selection, Slurm job configuration, interactive versus batch usage, and container-based runtimes. It also points to external guides and references that can be adapted for use on Sockeye.

Sockeye GPU Nodes

Node Type	Nodes	GPUs per Node	Partition	CPU	RAM per Node	Local Scratch	Notes
GPU16	6	4	Interactive	2 x Intel Xeon Silver 4116 (2.1 GHz)	192GB DDR4-2666 ECC	1.9TB NVMe SSD	Smaller models, smaller context, lighter workloads
GPU32	44	4	Compute	2 x Intel Xeon Silver 4216 (2.1 GHz)	192GB DDR4-2666 ECC	1.9TB NVMe SSD	Larger models, larger context, more headroom

GPU Ollama and OpenWebUI (LLM Inference)

> Use Ollama when you want a simple way to run a model locally for interactive testing and lightweight deployments (pull/manage models, run a local inference endpoint). Use OpenWebUI when you want a user-friendly chat interface for demos, internal use, or quick evaluation without building a front end. This setup is ideal for single-node inference, rapid iteration, and sharing a basic chat UI; for high-throughput, multi-GPU serving, or production-scale APIs, a dedicated serving stack (e.g., vLLM/TGI) is usually a better fit.

Use Apptainer to pull the OpenWebUI with Ollama image from Docker Hub. Choose the tag/version you want and save it as a .sif file.

Container preparation

```
module load gcc apptainer

mkdir -p /arc/project/<alloc-code>/containers

cd /arc/project/<alloc-code>/containers
apptainer pull openwebui.sif docker://ghcr.io/open-webui/open-webui:ollama
```

Prepare your workspace: create the directories we'll use for this workflow

Workspace preparation

```
# Writable workspace
export WEBUI_DATA="/scratch/<alloc-code>/<user>/openwebui-data"

# Container image location
export WEBUI_IMG="/arc/project/<alloc-code>/containers/openwebui.sif"

# Create required directories
mkdir -p "$WEBUI_DATA"                # workflow root
mkdir -p "$WEBUI_DATA/models"         # Ollama models (bind -> /root/.ollama)
mkdir -p "$WEBUI_DATA/static"         # REQUIRED: writable static override for OpenWebUI
mkdir -p "$WEBUI_DATA/hf/hub"         # HF cache (if used by the container)
mkdir -p "$WEBUI_DATA/logs"           # optional: keep logs organized

# Initialize the WebUI secret key (first run only)
if [ ! -f "$WEBUI_DATA/.webui_key" ]; then
  umask 077
  openssl rand -hex 32 > "$WEBUI_DATA/.webui_key"
fi
```

Because compute nodes don't have internet access, you must download models from a login node (or use a pre-downloaded model). A simple approach is to use the Ollama client included in openwebui.sif to pull models directly into your model directory.

Download models with Ollama

```
export OLLAMA_MODELS="$WEBUI_DATA/models"

# Start Ollama (background)
apptainer exec \
  -B "$OLLAMA_MODELS:/root/.ollama" \
  --env OLLAMA_MODELS=/root/.ollama \
  --env OLLAMA_HOST=127.0.0.1:11434 \
  "$WEBUI_IMG" \
  ollama serve > "$WEBUI_DATA/logs/ollama.log" 2>&1 &

# Monitor the log to verify Ollama started
tail -f $WEBUI_DATA/logs/ollama.log

# Pull a test model (example)
apptainer exec \
  -B "$OLLAMA_MODELS:/root/.ollama" \
  --env OLLAMA_MODELS=/root/.ollama \
  --env OLLAMA_HOST=http://127.0.0.1:11434 \
  "$WEBUI_IMG" \
  ollama pull tinyllama:latest
```

Once your models have been downloaded, create the ollama-webui.sh batch script below (update the SBATCH directives, paths, and port as needed). When the job starts, it will write a connection guide to connect-$SLURM_JOB_ID.txt in your submission directory ($SLURM_SUBMIT_DIR). Open that .txt file and follow the SSH port-forwarding steps to access the WebUI in your browser.

ollama-webui.sh

```
#!/bin/bash
#SBATCH --job-name=ollama
#SBATCH --account=<alloc-code>-gpu      # Add your GPU account
#SBATCH --time=02:00:00
#SBATCH --nodes=1
#SBATCH --gpus=1                        # Single GPU inferance
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4               # Enough for web server + tokenization
#SBATCH --mem=16G                       # Fit for UI + sqlite + cache; models live in GPU VRAM
#SBATCH --output=%x-%j.out
#SBATCH --error=%x-%j.err
#SBATCH --mail-user=<your-email>
#SBATCH --mail-type=ALL

source ~/.bashrc

module load gcc apptainer

WEBUI_PORT=18080   # Local browser port
WEBUI_DATA="/scratch/<alloc-code>/$USER/openwebui-data"
WEBUI_IMG="/arc/project/<alloc-code>/containers/openwebui.sif"
NODE="$(hostname -s)"

pick_port() {
  local start="$1"
  local end="$2"
  local port
  for port in $(shuf -i "${start}-${end}"); do
    if ! ss -lnt | awk '{print $4}' | grep -q ":${port}$"; then
      echo "$port"
      return 0
    fi
  done
  return 1
}

WEBUI_NODE_PORT="$(pick_port 8080 8999)"   # free OpenWebUI port on compute node
OLLAMA_PORT="$(pick_port 11434 11999)"     # free Ollama port on compute node

# UI Connection
cat > "${SLURM_SUBMIT_DIR}/connect-${SLURM_JOB_ID}.txt" <<EOF
============================================================

1) From your PC
ssh $USER@sockeye.arc.ubc.ca -L ${WEBUI_PORT}:127.0.0.1:${WEBUI_PORT}

2) Then from the login node in Step 1:
ssh $USER@$NODE -L ${WEBUI_PORT}:127.0.0.1:${WEBUI_NODE_PORT}

Then use a browser to open:
  http://127.0.0.1:${WEBUI_PORT}
EOF

# Start Open WebUI and Ollama
apptainer exec --nv --writable-tmpfs --pwd /app/backend \
  --env PORT="${WEBUI_NODE_PORT}" \
  --env OLLAMA_HOST="127.0.0.1:${OLLAMA_PORT}" \
  --env WEBUI_SECRET_KEY="$(cat "$WEBUI_DATA/.webui_key")" \
  --env OLLAMA_BASE_URL="http://127.0.0.1:${OLLAMA_PORT}" \
  --env OLLAMA_MODELS="/root/.ollama" \
  --env HF_HOME="/app/backend/data/hf" \
  --env HF_HUB_CACHE="/app/backend/data/hf/hub" \
  -B "$WEBUI_DATA:/app/backend/data" \
  -B "$WEBUI_DATA/.webui_key:/app/backend/.webui_key" \
  -B "$WEBUI_DATA/static:/app/backend/open_webui/static" \
  -B "$WEBUI_DATA/models:/root/.ollama" \
  "$WEBUI_IMG" \
  bash -c './start.sh'
```

OpenWebUI + Ollama container flags

Flag	Description
`WEBUI_SECRET_KEY=...`	Stable secret for consistent OpenWebUI sessions and config across runs.
`PORT="$WEBUI_NODE_PORT"`	Run OpenWebUI on a dynamically selected free port on the compute node.
`OLLAMA_HOST="127.0.0.1:${OLLAMA_PORT}"`	Bind Ollama to a dynamically selected free local port on the compute node.
`OLLAMA_BASE_URL="http://127.0.0.1:${OLLAMA_PORT}"`	Point OpenWebUI to the local Ollama API on the selected port.
`OLLAMA_MODELS="/root/.ollama"`	Model location inside the container.
`HF_HOME=... / HF_HUB_CACHE=...`	Keep Hugging Face cache in the persisted data location if needed.
`-B "$WEBUI_DATA:/app/backend/data"`	Persist WebUI state such as the database, config, and uploads.
`-B "$WEBUI_DATA/static:/app/backend/open_webui/static"`	Mount writable static assets outside the container if needed.
`-B "$WEBUI_DATA/models:/root/.ollama"`	Persist Ollama models outside the container.

vLLM with Multi GPU

> Use vLLM when you need fast, scalable LLM serving - especially for many concurrent users, high token throughput, and production-style APIs. It's a strong fit for hosting a model behind an OpenAI-compatible endpoint, running multi-GPU inference, and keeping latency reasonable under load (e.g., a team chatbot, an app backend, or a service that handles lots of requests). If you're only doing occasional interactive testing or a lightweight demo, Ollama/OpenWebUI is usually simpler; if you need reliability, throughput, and scale, vLLM is the better choice.

Prepare your workspace: Pick a persistent directory in your allocation for this workflow. We'll store the Hugging Face cache, downloaded models, and logs under this path so the compute job can run offline.

Workspace preparation

```
# Writable workspace
export VLLM_DATA="/scratch/<alloc-code>/<user>/vllm-data"

# Create required directories
mkdir -p "$VLLM_DATA"                # workflow root
mkdir -p "$VLLM_DATA/home"           # container HOME for vLLM/OpenWebUI
mkdir -p "$VLLM_DATA/models"         # downloaded models (MODEL_DIR will point here)
mkdir -p "$VLLM_DATA/hf/hub"         # Hugging Face hub cache
mkdir -p "$VLLM_DATA/hf/assets"      # Hugging Face assets cache
mkdir -p "$VLLM_DATA/logs"           # vLLM/OpenWebUI logs
mkdir -p "$VLLM_DATA/tmp"            # temporary runtime files (optional)
```

Download the model into "$VLLM_DATA/models" and point your batch script to the same MODEL_DIR. Example, this downloads Qwen2.5-7B-Instruct.

Download models from Hugging Face

```
# Download from a login node (recommended) or interactive GPU node with the http_proxy module loaded

export HF_HOME="$VLLM_DATA/hf"

export MODEL_REPO="Qwen/Qwen2.5-7B-Instruct"
export MODEL_DIR="$VLLM_DATA/models/Qwen/Qwen2.5-7B-Instruct"

huggingface-cli download "$MODEL_REPO" --local-dir "$MODEL_DIR"
```

Use Apptainer to pull the OpenAI vLLM image from Docker Hub. Choose the tag/version you want and save it as a *.sif* file.

Container preparation

```
module load gcc apptainer

cd /arc/project/<alloc-code>/containers
apptainer pull vllm.sif docker://vllm/vllm-openai
```

Create vllm-webui.sh and update the paths for VLLM_DATA, MODEL_DIR, and the container images. This job starts vLLM on the allocated node (multi-GPU tensor parallel), waits for the API health check, then starts OpenWebUI configured to use the local vLLM OpenAI-compatible endpoint. When the job starts, it writes a connection guide to connect-$SLURM_JOB_ID.txt in $SLURM_SUBMIT_DIR.

vllm-webui.sh

```
#!/bin/bash
#SBATCH --job-name=vllm-webui
#SBATCH --account=<alloc-code>-gpu
#SBATCH --time=02:00:00
#SBATCH --nodes=1
#SBATCH --gpus=2                      # 2 GPUs for tensor parallel
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8             # Allow CPU headroom for tokenization + request handling + WebUI
#SBATCH --mem=32G                     # Provide extra RAM buffer for vLLM + WebUI + caches/logs
#SBATCH --output=%x-%j.out
#SBATCH --error=%x-%j.err

source ~/.bashrc
module load gcc apptainer

# Paths (must match your workspace setup)
export VLLM_DATA="/scratch/<alloc-code>/<user>/vllm-data"
export MODEL_DIR="$VLLM_DATA/models/Qwen/Qwen2.5-7B-Instruct"
export VLLM_IMG="/arc/project/<alloc-code>/containers/vllm.sif"

# Reuse your existing OpenWebUI paths from the earlier section
export WEBUI_DATA="/scratch/<alloc-code>/<user>/openwebui-data"
export WEBUI_IMG="/arc/project/<alloc-code>/containers/openwebui.sif"

NODE="$(hostname -s)"

pick_port() {
  local start="$1"
  local end="$2"
  local port
  for port in $(shuf -i "${start}-${end}"); do
    if ! ss -lnt | awk '{print $4}' | grep -q ":${port}$"; then
      echo "$port"
      return 0
    fi
  done
  return 1
}

WEBUI_PORT="$(pick_port 3000 3999)"   # pick a free WebUI port
VLLM_PORT="$(pick_port 18000 18999)"  # pick a free vLLM port

# Two-step SSH tunnel to connect-$SLURM_JOB_ID.txt
cat > "$SLURM_SUBMIT_DIR/connect-${SLURM_JOB_ID}.txt" <<EOF
1. From your PC:
ssh -L ${WEBUI_PORT}:127.0.0.1:${WEBUI_PORT} -L ${VLLM_PORT}:127.0.0.1:${VLLM_PORT} $USER@sockeye.arc.ubc.ca

2. Then from the login node in Step 1:
ssh -L ${WEBUI_PORT}:127.0.0.1:${WEBUI_PORT} -L ${VLLM_PORT}:127.0.0.1:${VLLM_PORT} $NODE

Then use a browser to open:
http://127.0.0.1:${WEBUI_PORT}
EOF
```

```
# Start vLLM using 2 GPUs (tensor parallel)
nohup apptainer exec --nv --home "$VLLM_DATA/home" \
  --env HF_HUB_OFFLINE=1 \
  --env TRANSFORMERS_OFFLINE=1 \
  --env TORCH_COMPILE_DISABLE=1 \
  --env TORCHINDUCTOR_DISABLE=1 \
  --env TRITON_CACHE_DIR="$SLURM_TMPDIR/triton" \
  --env XDG_CACHE_HOME="$SLURM_TMPDIR/xdg" \
  -B "$VLLM_DATA/hf:/root/.cache/huggingface" \
  -B "$VLLM_DATA/tmp:/tmp" \
  -B "$MODEL_DIR:$MODEL_DIR" \
  "$VLLM_IMG" \
  vllm serve "$MODEL_DIR" \
    --host 127.0.0.1 --port "$VLLM_PORT" \
    --tensor-parallel-size 2 \
    --max-model-len 4096 \
    --max-num-seqs 8 \
    --enforce-eager \
    --served-model-name "Qwen2.5-7B-Instruct" \
  > "$VLLM_DATA/logs/vllm.log" 2>&1 &

# Wait for vLLM to become healthy before starting OpenWebUI
for i in $(seq 1 120); do
  curl -s "http://127.0.0.1:${VLLM_PORT}/health" >/dev/null && break
  sleep 1
done

# Start OpenWebUI and point it at the local vLLM endpoint
apptainer exec --nv --pwd /app/backend \
  --env PORT="$WEBUI_PORT" \
  --env HOST=127.0.0.1 \
  --env WEBUI_SECRET_KEY="$(cat "$WEBUI_DATA/.webui_key")" \
  --env USE_OLLAMA=0 \
  --env ENABLE_OLLAMA=0 \
  --env OPENAI_API_BASE_URL="http://127.0.0.1:${VLLM_PORT}/v1" \
  --env OPENAI_API_KEY="EMPTY" \
  --env HF_HOME="/app/backend/data/hf" \
  --env HF_HUB_CACHE="/app/backend/data/hf/hub" \
  --env TRANSFORMERS_CACHE="/app/backend/data/hf/transformers" \
  -B "$WEBUI_DATA:/app/backend/data" \
  -B "$WEBUI_DATA/static:/app/backend/open_webui/static" \
  -B "$VLLM_DATA/hf:/app/backend/data/hf" \
  -B "$VLLM_DATA/tmp:/tmp" \
  "$WEBUI_IMG" \
  bash -lc "/app/backend/start.sh" \
 > "$VLLM_DATA/logs/openwebui.log" 2>&1 &

wait
```

vLLM container flags

Flag	Description
`--host 127.0.0.1`	Bind vLLM to localhost only so it is accessed through SSH tunneling.
`--port "$VLLM_PORT"`	Run vLLM on a free selected port on the compute node.
`HF_HUB_OFFLINE=1`	Use pre-downloaded model. Change to 0 and load http_proxy module to access huggingface.co
`TRANSFORMERS_OFFLINE=1`	Use pre-downloaded model. Change to 0 and load http_proxy module to access huggingface.co
`TRITON_CACHE_DIR="$SLURM_TMPDIR/triton"`	Store Triton cache on node-local temporary storage.
`XDG_CACHE_HOME="$SLURM_TMPDIR/xdg"`	Store miscellaneous cache files on node-local temporary storage.
`--home "$VLLM_DATA/home"`	Use a writable HOME in your workspace instead of writing inside the container filesystem.
`-B "$VLLM_DATA/hf:/root/.cache/huggingface"`	Reuse Hugging Face cache across jobs and keep writes out of the container filesystem.

`-B "$VLLM_DATA/tmp:/tmp"`	Use your workspace temp directory inside the container.
`--tensor-parallel-size 2`	Split the model across 2 GPUs and match `#SBATCH --gpus=2`.
`--max-model-len 4096`	Limit context length to help control VRAM use.
`--max-num-seqs 8`	Limit concurrent sequences to balance throughput and memory use.
`--enforce-eager`	Disables CUDA graph capture to reduce VRAM use. Remove for better throughput.
`--served-model-name "Qwen2.5-7B-Instruct"`	Set the model name exposed to clients and the UI.
`> "$VLLM_DATA/logs/vllm.log" 2>&1 &`	Write logs to file and run vLLM in the background.

OpenWebUI container flags

Flag	Description
`HOST=127.0.0.1`	Bind WebUI to localhost only so it is accessed through SSH tunneling.
`PORT="$WEBUI_PORT"`	Run OpenWebUI on a free selected port on the compute node.
`OPENAI_API_BASE_URL="http://127.0.0.1:${VLLM_PORT}/v1"`	Point OpenWebUI to the local vLLM endpoint.
`OPENAI_API_KEY="EMPTY"`	Placeholder value for the local vLLM backend.
`USE_OLLAMA=0`	Disable Ollama integration because this setup uses vLLM.
`ENABLE_OLLAMA=0`	Explicitly keep Ollama disabled.
`WEBUI_SECRET_KEY="$(cat "$WEBUI_DATA/.webui_key")"`	Use a stable secret key for consistent WebUI sessions.
`-B "$WEBUI_DATA:/app/backend/data"`	Persist the WebUI database, configuration, and uploads outside the container.
`-B "$WEBUI_DATA/static:/app/backend/open_webui/static"`	Mount optional static assets outside the container.
`-B "$VLLM_DATA/hf:/app/backend/data/hf"`	Reuse the same Hugging Face cache location inside WebUI.
`-B "$VLLM_DATA/tmp:/tmp"`	Use your workspace temp directory inside the container.
`> "$VLLM_DATA/logs/openwebui.log" 2>&1 &`	Write logs to file and run OpenWebUI in the background.

> **Other UI Options**
>
> OpenWebUI is one option for a lightweight chat front-end. Two common alternatives are AnythingLLM (often used for chat + document/RAG workflows) and LibreChat (multi-provider chat UI with strong OpenAI-style support). If you choose an alternative UI, the key requirement is support for connecting to an OpenAI-compatible API base URL (such as a local vLLM endpoint).

Interactive vLLM with Python

Start an interactive GPU session. Adjust the account, time, memory, and GPU count as needed for your model and workload. (a single GPU on an interactive node has 16GB VRAM)

Request interactive_gpu

```
salloc --account=<alloc-code>-gpu -N 1 -n 8 --mem=32G --gpus=4 --time=02:00:00 -p interactive_gpu
```

Update VLLM_DATA and MODEL_DIR to match where your model is stored.

$VLLM_DATA & $MODEL_DIR

```
module load gcc apptainer python

export VLLM_DATA="/scratch/<alloc-code>/<user>/vllm-data"
export MODEL_DIR="$VLLM_DATA/models/Qwen/Qwen2.5-7B-Instruct"
```

Launch the vLLM server inside the container

Start vLLM

```
nohup apptainer exec --nv --cleanenv --home "$VLLM_DATA/home" \
  --env HF_HUB_OFFLINE=1 \
  --env TRANSFORMERS_OFFLINE=1 \
  --env TORCH_COMPILE_DISABLE=1 \
  --env TORCHINDUCTOR_DISABLE=1 \
  --env TRITON_CACHE_DIR="$SLURM_TMPDIR/triton" \
  --env XDG_CACHE_HOME="$SLURM_TMPDIR/xdg" \
  -B "$VLLM_DATA/hf:/root/.cache/huggingface" \
  -B "$VLLM_DATA/tmp:/tmp" \
  -B "$MODEL_DIR:$MODEL_DIR" \
  /arc/project/<alloc-code>/containers/vllm.sif \
  vllm serve "$MODEL_DIR" \
    --host 127.0.0.1 --port 18000 \
    --tensor-parallel-size 4 \
    --max-model-len 4096 \
    --max-num-seqs 8 \
    --enforce-eager \
  > "$VLLM_DATA/logs/vllm.log" 2>&1 &
```

Use this snippet to send a test chat request directly to the running vLLM server

Python test request script

```
python3 - <<'PY'
import requests
BASE="http://127.0.0.1:18000/v1"
MODEL="/scratch/<alloc-code>/<user>/vllm-data/models/Qwen/Qwen2.5-7B-Instruct"

docs=["Write a 120+ word paragraph about UBC in Vancouver, mentioning research computing."]

for i,text in enumerate(docs,1):
    payload={"model":MODEL,"messages":[{"role":"user","content":text}],"temperature":0.2,"max_tokens":250}
    r=requests.post(f"{BASE}/chat/completions",json=payload,timeout=120)
    r.raise_for_status()
    j=r.json()
    out=j["choices"][0]["message"]["content"]
    print(f"\n--- DOC {i} ---\n{out}\n")
PY
```

Common methods for monitoring and cleanup while testing

Monitoring

```
# Monitor log(s)
tail -f "$VLLM_DATA/logs/vllm.log"

# Clean PID/environment - instances where you need to restart
pkill -u "$USER" -f 'vllm serve' || true
```

Model Fine Tuning Basics

RAG vs. Fine-Tuning

Fine-tuning “teaches” a model by training it on your own data so it reliably answers in a specific style or follows a consistent workflow, while retrieval-augmented generation (RAG) leaves the model unchanged and instead pulls relevant info from your knowledge sources at question time (PDFs, web pages, wikis, docs, databases, etc.) to ground the response. RAG is usually better when information changes or must stay up to date - for example, a customer-support bot that answers from the latest help center articles, or an internal assistant that references current company policies and procedures. Fine-tuning is usually better when the facts are relatively stable and you want the model to respond more consistently-for example, generating replies in a brand voice, producing structured outputs like “problem > diagnosis > next steps,” or handling a repeating set of Q&A with the same wording and tone. This guide focuses on fine-tuning a small open-source model using Q&A pairs derived from a HTML, covering dataset prep, cleaning, LoRA/QLoRA training, and basic evaluation.

In this example, we'll fine-tune a small instruct model using QLoRA (4-bit base model + LoRA adapters) and produce an adapter directory you can load at inference time

Workspace preparation

```
export VLLM_DATA="/scratch/<alloc-code>/<user>/vllm-data"
export HF_HOME="$VLLM_DATA/hf"

export FT_DATA="/scratch/<alloc-code>/<user>/finetune-data"
mkdir -p "$FT_DATA"/{data,output,logs,tmp}
```

Download a model from HuggingFace

Download models from Hugging Face

```
# Run from a login node (recommended) or interactive GPU node with the http_proxy module loaded

export MODEL_DIR="$VLLM_DATA/models/Qwen/Qwen2.5-1.5B-Instruct"
huggingface-cli download "Qwen/Qwen2.5-1.5B-Instruct" --local-dir "$MODEL_DIR"
```

Use Apptainer to download a PyTorch container for training

$FT_IMG

```
module load gcc apptainer

cd /arc/project/<alloc-code>/containers
apptainer pull pytorch.sif docker://nvcr.io/nvidia/pytorch:24.01-py3
export FT_IMG="/arc/project/<alloc-code>/containers/pytorch.sif"
```

Create a python virtual environment and install prerequisits

$FT_DATA/venv

```
module load python
export VENV_DIR="$FT_DATA/venv"
python3 -m venv "$VENV_DIR"
source "$VENV_DIR/bin/activate"

pip install --upgrade pip
pip install "transformers>=4.45" datasets accelerate "peft>=0.10" "trl>=0.8,<0.12" "bitsandbytes>=0.43"
safetensors sentencepiece rich
```

Prepare a training script. This example will be based on some content from this AI/ML Resorces Hub page

$FT_DATA/data/train.jsonl

```
{"messages":[{"role":"system","content":"You are an ARC support assistant. Be concise and accurate."},{"role":"
user","content":"How do I check GPU usage on my job?"},{"role":"assistant","content":"SSH to the compute node
listed in your connect file, then run nvidia-smi for a snapshot or nvtop for a live view."}]}
{"messages":[{"role":"system","content":"You are an ARC support assistant. Be concise and accurate."},{"role":"
user","content":"Where should I store container images?"},{"role":"assistant","content":"Store container images
under /arc/project/<alloc-code>/containers. Store runtime data such as models and logs under /scratch/<alloc-
code>/$USER."}]}
{"messages":[{"role":"system","content":"You are an ARC support assistant. Be concise and accurate."},{"role":"
user","content":"When should I use vLLM instead of Ollama?"},{"role":"assistant","content":"Use Ollama for
quick single-user testing. Use vLLM when you need multi-GPU inference, higher throughput, or an OpenAI-
compatible API endpoint."}]}
{"messages":[{"role":"system","content":"You are an ARC support assistant. Be concise and accurate."},{"role":"
user","content":"Can I download a HuggingFace model from a compute node?"},{"role":"assistant","content":"Yes.
Load the http_proxy module first — huggingface.co is in the proxy allowlist. For batch jobs, pre-download
before submitting to avoid using GPU time on network transfers."}]}
{"messages":[{"role":"system","content":"You are an ARC support assistant. Be concise and accurate."},{"role":"
user","content":"How do I connect to the WebUI after my job starts?"},{"role":"assistant","content":"Open the
connect-$SLURM_JOB_ID.txt file in your submission directory. It contains the SSH tunnel commands and the
localhost URL to open in your browser."}]}
```

Additionally create a small validation set so training can run evaluation steps

$FT_DATA/data/val.jsonl

```
{"messages":[{"role":"system","content":"You are an ARC support assistant. Be concise and accurate."},{"role":"
user","content":"What is the difference between GPU16 and GPU32 nodes?"},{"role":"assistant","content":"GPU16
nodes have 16GB VRAM per GPU and are suitable for smaller models and lighter workloads. GPU32 nodes have 32GB
VRAM and are better for larger models or longer context lengths."}]}
{"messages":[{"role":"system","content":"You are an ARC support assistant. Be concise and accurate."},{"role":"
user","content":"Can I pull a container image from a compute node?"},{"role":"assistant","content":"No. Docker
Hub and other container registries are not in the proxy allowlist. Run apptainer pull on a login node and save
the .sif file to /arc/project/<alloc-code>/containers."}]}
```

Prepare a training script

$FT_DATA/train_qlora.py

```
import os
import torch
from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    TrainingArguments,
    BitsAndBytesConfig,
)
from trl import SFTTrainer
from peft import LoraConfig

MODEL_DIR  = os.environ["MODEL_DIR"]
TRAIN_FILE = os.environ["TRAIN_FILE"]
VAL_FILE   = os.environ["VAL_FILE"]
OUT_DIR    = os.environ["OUT_DIR"]

tokenizer = AutoTokenizer.from_pretrained(MODEL_DIR, use_fast=True)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

# Load JSONL with chat-style messages
ds = load_dataset("json", data_files={"train": TRAIN_FILE, "validation": VAL_FILE})

# Convert messages -> plain text
def render_chat(example):
```

```
    text = tokenizer.apply_chat_template(
        example["messages"],
        tokenize=False,
        add_generation_prompt=False,
    )
    return {"text": text}

ds = ds.map(render_chat)

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True,
)

model = AutoModelForCausalLM.from_pretrained(
    MODEL_DIR,
    torch_dtype=torch.float16,
    quantization_config=bnb_config,
    device_map="auto",
)

lora = LoraConfig(
    r=16,
    lora_alpha=32,
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"],
)

args = TrainingArguments(
    output_dir=OUT_DIR,
    per_device_train_batch_size=1,
    per_device_eval_batch_size=1,
    gradient_accumulation_steps=16,
    learning_rate=2e-4,
    num_train_epochs=3,
    logging_steps=5,
    save_steps=20,
    eval_steps=20,
    eval_strategy="steps",
    save_total_limit=2,
    fp16=True,
# W&B logging available — set to "wandb" and load http_proxy module to enable
    report_to="none",
)

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=ds["train"],
    eval_dataset=ds["validation"],
    peft_config=lora,
    args=args,
    dataset_text_field="text",
    max_seq_length=1024,
)

trainer.train()
trainer.save_model(OUT_DIR)
print("Saved adapter to:", OUT_DIR)
```

Prepare a slurm job script

$FT_DATA/finetune-qlora.sh

```
#!/bin/bash
#SBATCH --job-name=qlora
#SBATCH --account=<alloc-code>-gpu
#SBATCH --time=02:00:00
#SBATCH --nodes=1
#SBATCH --gpus=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --output=%x-%j.out
#SBATCH --error=%x-%j.err

# The http_proxy module enables huggingface.co access for tokenizer/config fetches if needed

module load gcc apptainer python http_proxy

export VLLM_DATA="/scratch/<alloc-code>/<user>/vllm-data"
export FT_DATA="/scratch/<alloc-code>/<user>/finetune-data"
export HF_HOME="$VLLM_DATA/hf"

export MODEL_DIR="$VLLM_DATA/models/Qwen/Qwen2.5-1.5B-Instruct"
export TRAIN_FILE="$FT_DATA/data/train.jsonl"
export VAL_FILE="$FT_DATA/data/val.jsonl"
export OUT_DIR="$FT_DATA/output/adapter-${SLURM_JOB_ID}"

export VENV_DIR="$FT_DATA/venv"
export FT_IMG="/arc/project/<alloc-code>/containers/pytorch.sif"

# Node-local caches
export XDG_CACHE_HOME="$SLURM_TMPDIR/xdg"
export TRITON_CACHE_DIR="$SLURM_TMPDIR/triton"
mkdir -p "$XDG_CACHE_HOME" "$TRITON_CACHE_DIR"


apptainer exec --nv \
  -B "$VLLM_DATA:$VLLM_DATA" \
  -B "$FT_DATA:$FT_DATA" \
  -B "$SLURM_TMPDIR:/tmp" \
  "$FT_IMG" \
  bash -lc "
    source '$VENV_DIR/bin/activate'
    python3 '$FT_DATA/train_qlora.py' 2>&1 | tee '$FT_DATA/logs/train-${SLURM_JOB_ID}.log'
  "
```

After the training has successfully completed, use an Interactive GPU session to test inference against the trained adapter and base model separately to observe the difference if output. The fine-tuned adapter should return a Sockeye-specific answer. The base model will likely respond more generically or verbosely

Request interactive_gpu

```
salloc --account=<alloc-code>-gpu -N 1 -n 8 --mem=32G --gpus=1 -p interactive_gpu
```

$ADAPTER_DIR

```
# In this case replace ${SLURM_JOB_ID} with the applicable JobID
export ADAPTER_DIR="$FT_DATA/output/adapter-${SLURM_JOB_ID}"
source "$FT_DATA/venv/bin/activate"
```

Use Python to test adapter inferencing interactively

Python adapter test

```
python3 - <<'PY'
import os
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
from peft import PeftModel

MODEL_DIR = os.environ["MODEL_DIR"]
ADAPTER_DIR = os.environ["ADAPTER_DIR"]

tok = AutoTokenizer.from_pretrained(MODEL_DIR, use_fast=True)
base = AutoModelForCausalLM.from_pretrained(
    MODEL_DIR,
    torch_dtype=torch.float16,
    device_map="auto",
)
model = PeftModel.from_pretrained(base, ADAPTER_DIR)
model.eval()

prompt = "How do I check GPU usage on my Sockeye job?"
inputs = tok(prompt, return_tensors="pt").to(model.device)
with torch.no_grad():
    out = model.generate(
        **inputs,
        max_new_tokens=200,
        do_sample=False,
        pad_token_id=tok.eos_token_id,
    )
print(out)
PY
```

For the base-model comparison, use the same test script and prompt, but omit ADAPTER_DIR and the PeftModel.from_pretrained(...) step so generation runs directly from the base model.

Base model test

```
python3 - <<'PY'
# Same as adapter test but without PEFT
import os
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

MODEL_DIR = os.environ["MODEL_DIR"]

tok = AutoTokenizer.from_pretrained(MODEL_DIR, use_fast=True)
base = AutoModelForCausalLM.from_pretrained(
    MODEL_DIR,
    torch_dtype=torch.float16,
    device_map="auto",
)
base.eval()

prompt = "How do I check GPU usage on my Sockeye job?"
inputs = tok(prompt, return_tensors="pt").to(base.device)
with torch.no_grad():
    out = base.generate(
        **inputs,
        max_new_tokens=200,
        do_sample=False,
        pad_token_id=tok.eos_token_id,
    )
print(out)
PY
```

Monitoring

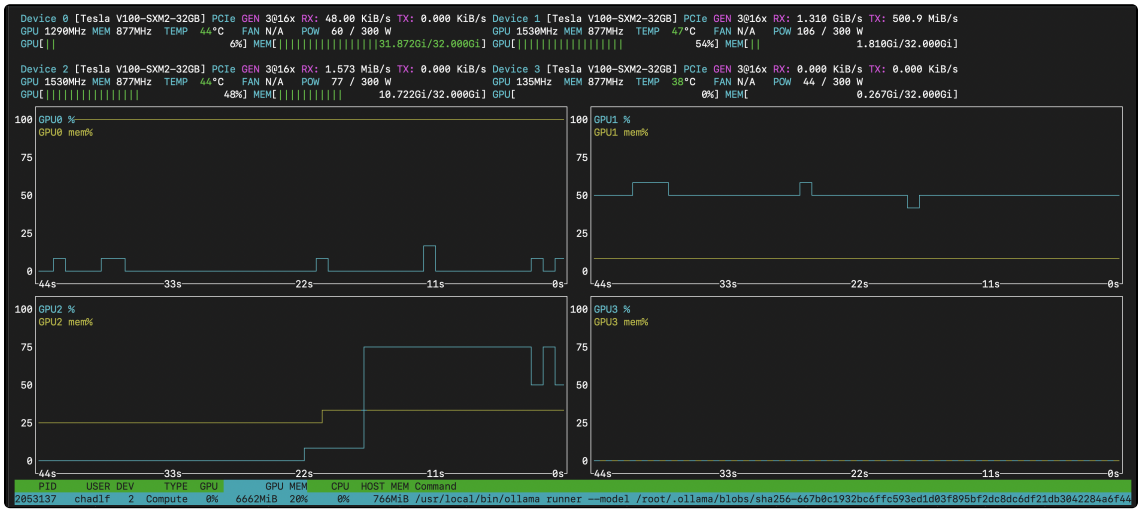
When your job starts, the script writes a connection helper file named `connect-$SLURM_JOB_ID.txt` in your submission directory (`$SLURM_SUBMIT_DIR)`. Open this file to see the compute node hostname (e.g., se267) and the SSH tunnel steps for your running job.

```
ssh $USER@${NODE}
```

Once on the compute node, you can monitor GPU usage with tools such as nvidia-smi and/or nvtop.

NVTOP

nvtop provides a live, top-like view of GPU utilization and per-process GPU memory usage. You can install it in a Conda virtual environment and run it from within your job session.



NVIDIA-SMI

nvidia-smi is available on the GPU nodes by default and is the quickest way to confirm your job is using the GPU and to check GPU memory usage, utilization, and running processes.

```
nvidia-smi
```

External Guides and References

These links point to related documentation from other universities and research computing centers. They are meant to complement our notes above and show common patterns - requesting GPUs in Slurm, structuring job scripts, setting up PyTorch/TensorFlow environments, using containers, and scaling to multi-GPU/multi-node that you can adapt to Sockeye.

Alliance Canada - AI and Machine Learning: Practical guidance for running AI/ML workloads on Alliance clusters, including environment best practices.

Stanford Research Computing - PyTorch and Keras: Walkthrough of a complete GPU Slurm job, including a short training example in both PyTorch and TensorFlow/Keras.

University of Sheffield - GPUs and basic monitoring.

Oxford (Stats HPC) - PyTorch/TensorFlow conda and Slurm: Intro guide showing how to use PyTorch/TensorFlow Conda environments with an example Slurm workflow.

vLLM Documentation - Reference for serve flags, supported models, quantization options, and performance tuning.

Ollama Model Library - Search available models. Check VRAM requirements before pulling.

OpenWebUI Documentation - Configuration, environment variables, RAG setup, and pipeline options.

HuggingFace PEFT Documentation - Reference for LoRA, QLoRA, and other fine-tuning methods.

Is there something you'd like to see added to this page? Please email arc.support@ubc.ca